

Software Requirements Specification

For

Kolmogorov-Arnold Networks in Recurrent
Recommenders: Session-Based and Next-Basket
Prediction

6 October 2026

Prepared by

Specialization	SAP ID	Name
B.Tech CSE	500124297	Abhishek Yadav
B.Tech CSE	500121969	Saksham Agrawal
B.Tech CSE	500125147	Yash Dagar



AI Cluster
School Of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES,
DEHRADUN- 248 007. Uttarakhand

Table of Contents

Topic		Page No
Table of Content		2
Revision History		2
1	Introduction	4
	1.1 Purpose of the Project	4
	1.2 Target Beneficiary	4
	1.3 Project Scope	4
	1.4 References	5
2	Project Description	7
	2.1 Reference Algorithm	7
	2.2 Data/ Data structure	7
	2.3 SWOT Analysis	8
	2.4 Project Features	8
	2.5 User Classes and Characteristics	9
	2.6 Design and Implementation Constraints	9
	2.7 Design diagrams	10
	2.8 Assumption and Dependencies	15
3	System Requirements	15
	3.1 User Interface	15
	3.2 Software Interface	15
	3.3 Database Interface	16
	3.4 Protocols	16
4	Non-functional Requirements	16
	4.1 Performance requirements	16
	4.2 Security requirements	17
	4.3 Software Quality Attributes	17
5	Other Requirements	17
Appendix A: Glossary		18
Appendix B: Analysis Model		18
Appendix C: Issues List		18

Revision History

Date	Change	Reason for Changes	Mentor Signature
06-10-2026	Version 1.0: complete SRS	End-term submission of the major project	

1 INTRODUCTION

1.1 Purpose of the Project

Recommender systems rank the items of a large catalogue for a user. This project studies two sequential tasks: session-based recommendation, which predicts the next item clicked in an anonymous browsing session, and next-basket recommendation, which predicts the products in a customer's next grocery basket from their earlier baskets. Both tasks are usually modelled with gated recurrent units (GRUs): GRU4Rec for sessions and the basket-GRU of van Maasakkers et al. (2023) for baskets.

Kolmogorov-Arnold Networks (KANs) replace the fixed activation functions of a multi-layer perceptron (MLP) with a learnable univariate function on every edge. They are claimed to be more accurate per parameter and more interpretable than MLPs, but later studies found that these advantages mostly disappear when the two are compared at the same size.

Problem statement. Determine whether, and where, KAN layers improve GRU-based session and next-basket recommenders, under a protocol that allows a fair conclusion: standard leakage-free splits, verified implementations, parameter-matched MLP controls, and evaluation on three public datasets.

Motivation. The first draft of this project compared KAN and GRU models on a data split that leaked test sessions into training, selected models on the test set and used a B-spline routine that produced step functions. Its numbers could not support any conclusion. This system re-does the study so that both a positive and a negative result are trustworthy.

1.2 Target Beneficiary

- **Recommender-system researchers:** a controlled, reproducible answer to whether KAN layers help recurrent recommenders, with all code and results public.
- **E-commerce and online-grocery platforms:** evidence on which model to deploy, and an interpretable repeat-purchase model (KAN-NBR) whose behaviour can be explained to business users.
- **Students and educators:** a worked example of a fair model comparison, including the errors of a first draft and how they were fixed.
- **End users (shoppers):** indirectly, through better and more explainable recommendations that need no long-term tracking in the session setting.

1.3 Project Scope

The software is a research pipeline, not an end-user application. It downloads the datasets, preprocesses them with published protocols, trains every model variant, evaluates it once on a held-out test set, runs significance tests, and writes the tables and figures of the report. Its goals are:

- a correct, unit-tested KAN layer with four basis families (B-spline, Gaussian RBF, Chebyshev, group-rational);
- KAN layers in three placements (head, input, GRU cell) of the official GRU4Rec and of the basket-GRU, each with a parameter-matched MLP control;

- strong non-neural baselines (Pop, S-Pop, item-kNN, TopFreq variants, TIFU-KNN);
- KAN-NBR, an additive KAN over interpretable repeat-purchase features;
- three hardware profiles (Mac, UPES Blackwell GPU, Google Colab) selected by one variable.

Deliverables: the notebook `kan_rec.ipynb` and the UPES notebook `kan_rec_upes.ipynb`; the Python package `kanrec` with 41 unit tests; per-run result files; the project report (LaTeX and a single-file Overleaf version); the mid-term and end-term presentations; and this SRS. All are public at <https://github.com/yashdagar/kan-rec>. Out of scope: a production service, a graphical user interface and real-time serving.

1.4 References

1. Hidasi, B., Karatzoglou, A., Baltrunas, L., & Tikk, D. (2016). Session-based recommendations with recurrent neural networks. ICLR.
2. Hidasi, B., & Karatzoglou, A. (2018). Recurrent neural networks with top-k gains for session-based recommendations. CIKM, 843-852.
3. Hidasi, B., & Czapp, A. T. (2023). The effect of third party implementations on reproducibility. RecSys, 272-282.
4. Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., & Tan, T. (2019). Session-based recommendation with graph neural networks. AAAI, 346-353.
5. Hu, H., He, X., Gao, J., & Zhang, Z.-L. (2020). Modeling personalized item frequency information for next-basket recommendation. SIGIR, 1071-1080.
6. Li, M., Jullien, S., Ariannezhad, M., & de Rijke, M. (2023). A next basket recommendation reality check. ACM TOIS, 41(4), 116.
7. van Maasakkers, L., Fok, D., & Donkers, B. (2023). Next-basket prediction in a high-dimensional setting using gated recurrent units. Expert Systems with Applications, 212, 118795.
8. Liu, Z., Wang, Y., Vaidya, S., et al. (2025). KAN: Kolmogorov-Arnold networks. ICLR.
9. Yu, R., Yu, W., & Wang, X. (2024). KAN or MLP: A fairer comparison. arXiv:2407.16674.
10. Genet, R., & Inzirillo, H. (2024). TKAN: Temporal Kolmogorov-Arnold networks. arXiv:2405.07344.
11. Yang, X., & Wang, X. (2025). Kolmogorov-Arnold Transformer. ICLR.
12. IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
13. Official GRU4Rec (PyTorch): https://github.com/hidasib/GRU4Rec_PyTorch_Official
14. YooChoose (RecSys Challenge 2015): <https://www.kaggle.com/datasets/chadgostopp/recsys-challenge-2015>
15. Instacart Market Basket Analysis: <https://www.kaggle.com/c/instacart-market-basket-analysis>

16. Dunnhumby, The Complete Journey:

<https://www.kaggle.com/datasets/frtgnn/dunnhumby-the-complete-journey>

17. Next-basket GRU code:

<https://github.com/luukvanmaasakkers/nextbasketpredictionGRU>

18. TARS / tbp-next-basket: <https://github.com/GiulioRossetti/tbp-next-basket>

19. Project repository: <https://github.com/yashdagar/kan-rec>

2 PROJECT DESCRIPTION

2.1 Reference Algorithm

The reference algorithm is GRU4Rec (Hidasi et al. 2016, 2018) in its official PyTorch implementation. KAN layers are added to it, and to the basket-GRU, without changing the training procedure.

Algorithm 1: GRU4Rec training with KAN blocks

1. Sort training clicks by session and time; keep an offset array marking where each session starts.
2. Fill 48 mini-batch slots with the first 48 sessions; each slot advances one click per step.
3. For the current items, look up embeddings $x = E[i]$; optionally apply the input block $x' = x + f(\text{LN}(x))$.
4. Update the hidden state with the GRU cell (or the KAN-GRU cell, whose candidate state is $\tanh(\text{KAN}([x, r * h]))$).
5. Optionally apply the head block $h' = h + f(\text{LN}(h))$, then score the targets and 2048 shared negative items sampled by $\text{popularity}^{0.2}$ with the dot product $E h'$.
6. Minimise softmax cross-entropy with logQ correction using Adagrad (lr 0.07; added parameters lr 0.001).
7. When a session ends, reset its slot's hidden state and load the next session.
8. After each epoch compute validation Recall@20 and keep the best weights; evaluate once on the test set.

The KAN layer computes, for each output o , $y_o = \text{sum over inputs } i \text{ of } \phi_{oi}(x_i)$, where $\phi(x) = w * \text{silu}(x) + \text{sum}_k c_k B_k(x)$ and B_k is a fixed basis (cubic B-spline by the Cox-de Boor recursion, Gaussian RBF, Chebyshev polynomials of $\tanh(x)$, or a group-rational function). Each KAN block has an MLP control with the same number of parameters (within 2%).

Data structures

Table 1: Data structures used by the system.

Structure	Used for	Implementation
Offset array + item index array	session-parallel mini-batches	NumPy int64 arrays, on GPU blocks
Embedding matrix E (items x 480)	shared input and output item vectors	torch.nn.Parameter
List of baskets per user	basket histories	Python lists of int arrays (BasketData)
Index + offset lists	multi-hot baskets without dense vectors	torch.nn.EmbeddingBag
Sparse co-occurrence matrix	item-kNN and product clustering	SciPy CSR matrix
Feature matrix (pairs x 7)	KAN-NBR training pairs	float32 tensor
Result record	metrics, history, timing per run	JSON file + NPZ per-user arrays

2.2 Characteristic of Data

All data are secondary, public datasets downloaded from Kaggle; no primary data are collected. Table 2 summarises them after preprocessing.

Table 2: Datasets after preprocessing (Mac profile).

Property	YooChoose 1/64	Instacart	Dunnhumby
Source	RecSys Challenge 2015	Instacart Market Basket	The Complete Journey
Raw size	33.0 M clicks, 9.25 M sessions	3.4 M orders, 206,209 users	2.60 M rows, 2,500 households
Unit	click in a session	basket of a user	basket of a household
After preprocessing	139,628 sessions	20,620 users (10% sample)	2,478 households
Items	17,264	12,058 product groups	10,370
Mean length	4.1 clicks	16.2 baskets, 10.1 items	66.2 baskets, 8.7 items
Test targets	55,357 next clicks	10,310 baskets	1,239 baskets
Repeat share of target	-	0.60	0.53

Sampling

- YooChoose: the most recent 1/64 of training sessions (standard yoochoose1/64); sessions of the last day form the test set and the last day of training data the validation set.
- Instacart: a random 10% of users on the Mac profile (all users on the GPU profile); the last basket of each user is the target; users are split 50/50 into validation and test with a fixed seed.
- Dunnhumby: all households with at least 3 baskets, the 100 most recent baskets kept; same 50/50 split.

Statistical processing

- Support filters: items in at least 5 sessions (YooChoose) or bought at least 50 times (Dunnhumby).
- Instacart rare products (fewer than 500 purchases) merged greedily within their aisle by cosine similarity of purchase vectors.
- Paired Wilcoxon signed-rank tests on per-prediction or per-user metrics with Holm correction.

2.3 SWOT Analysis

Table 3: SWOT analysis.

Strengths	Weaknesses
Official GRU4Rec code and published protocols; parameter-matched controls; 41 unit tests; three datasets and two tasks; resumable runs.	KAN layers are slower to train; extra hyperparameters (basis, grid size); Mac results use one seed and a 10% Instacart sample.
Opportunities	Threats
Interpretable repeat-purchase models (KAN-NBR); larger runs on the UPES Blackwell GPU; KAN layers in attention-based recommenders.	KANs may show no advantage over MLPs; transformer recommenders dominate recent work; limited compute time for many variants.

Justification: the strengths address the exact weaknesses of the first draft (leakage, unverified code, no capacity control), so whatever the outcome, the comparison is credible. The main threat, a negative result for KAN, is mitigated by framing a rigorous negative result as a contribution and by KAN-NBR, which uses KANs where their interpretability matters.

2.4 Project Features

- **Profile switch:** one variable selects Mac, UPES or Colab settings, paths and device.

- **Data pipelines:** download, preprocessing and leakage assertions for three datasets.
- **KAN module:** four bases, residual block with zero-initialised output, matched MLP.
- **Model zoo:** GRU4Rec and basket-GRU with KAN or MLP head, input or cell; draft models; KAN-NBR.
- **Baselines:** Pop, S-Pop, item-kNN, G/P/GP-TopFreq, last basket, TIFU-KNN.
- **Evaluation:** Recall, MRR, NDCG, PHR, $P@|B|$, average rank, repeat/explore split.
- **Resumable experiments:** one result file per dataset, model and seed.
- **Reporting:** LaTeX tables, figures, significance tests, learned-function plots.

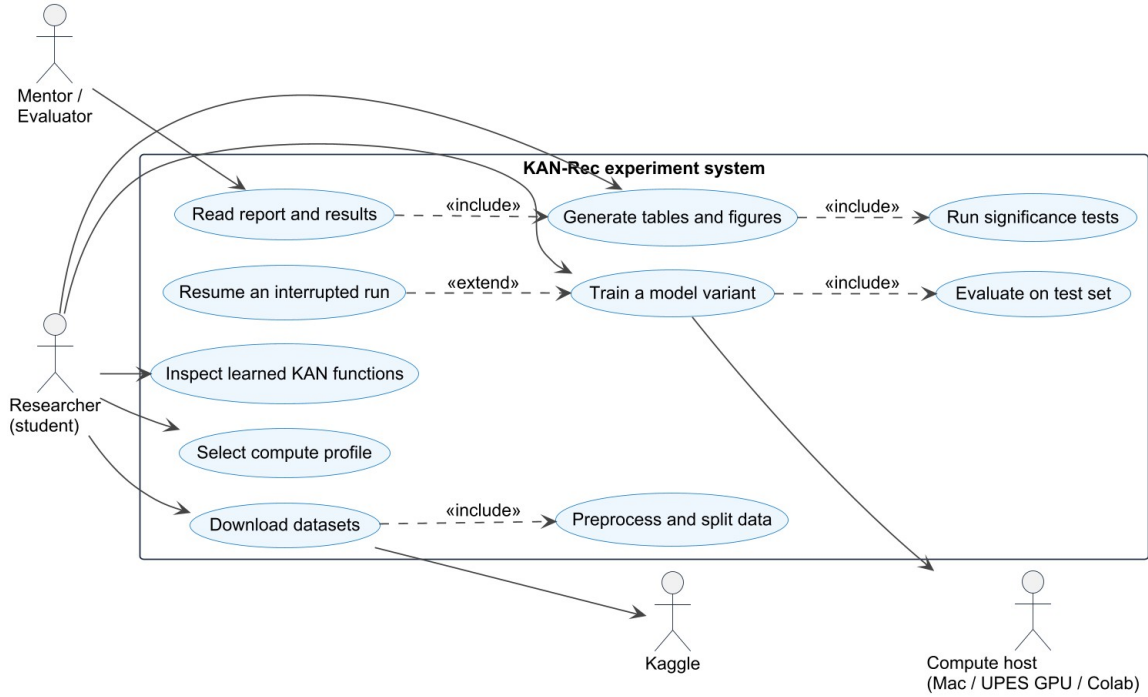


Figure 1: Level-2 use case diagram (Source: Authors' own work)

2.5 User Classes and Characteristics

Table 4: User classes.

User class	Characteristics	Main use
Researcher (project team)	Python and PyTorch skills; runs the notebook on all profiles	run experiments, extend models
Mentor / evaluator	reads results; may re-run selected cells	review report and results
External researcher	clones the public repository	reproduce or extend the study
Practitioner	data scientist at a retailer	reuse KAN-NBR or baselines on own data

2.6 Design and Implementation Constraints

- **Hardware:** Apple M1 Pro with 16 GB unified memory (Mac profile); PyTorch MPS has no float64, so all tensors are float32; memory is freed after every run. UPES profile: NVIDIA Blackwell GPU with CUDA. Colab: T4/L4 GPU with session time limits.
- **Timing:** one GRU4Rec epoch on YooChoose 1/64 takes about 75 s on the Mac; the full Mac run takes about two days, so every run must be resumable.

- **Interfaces:** official GRU4Rec code (academic licence, cloned at a pinned commit, not redistributed); kagglehub for downloads.
- **Technologies:** Python 3.12, PyTorch 2.x, NumPy, pandas, SciPy, Matplotlib, Jupyter, uv, LaTeX.
- **Parallel operation:** session-parallel mini-batches; GPU kernels; chunked evaluation to bound memory.
- **Standards:** PEP 8 style; unit tests with pytest; fixed random seeds; no test-set use for model selection.
- **Security:** the Kaggle API token stays in the user's home directory and is never committed.

2.7 Design Diagrams

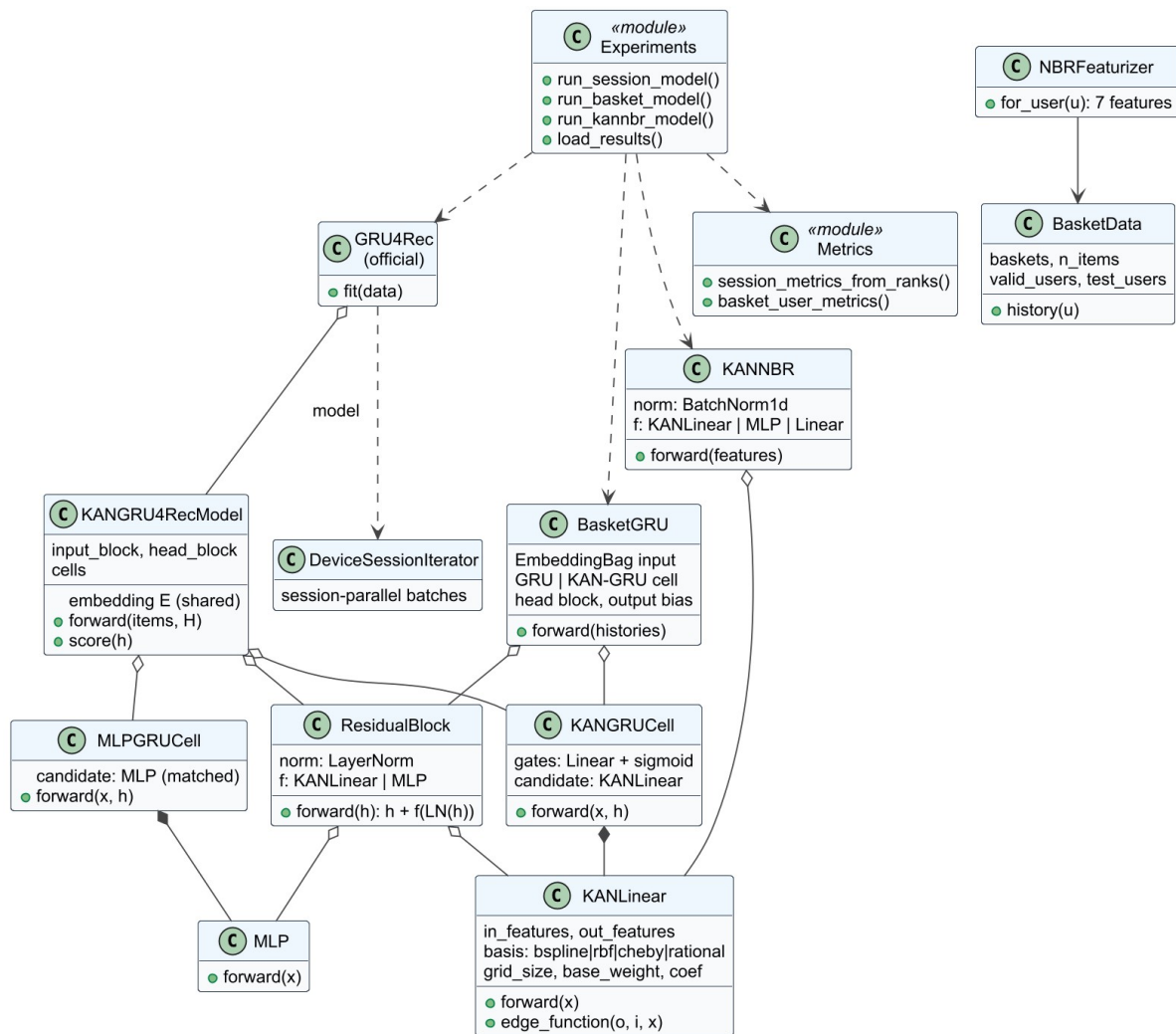


Figure 2: Class diagram of the kanrec package (Source: Authors' own work)

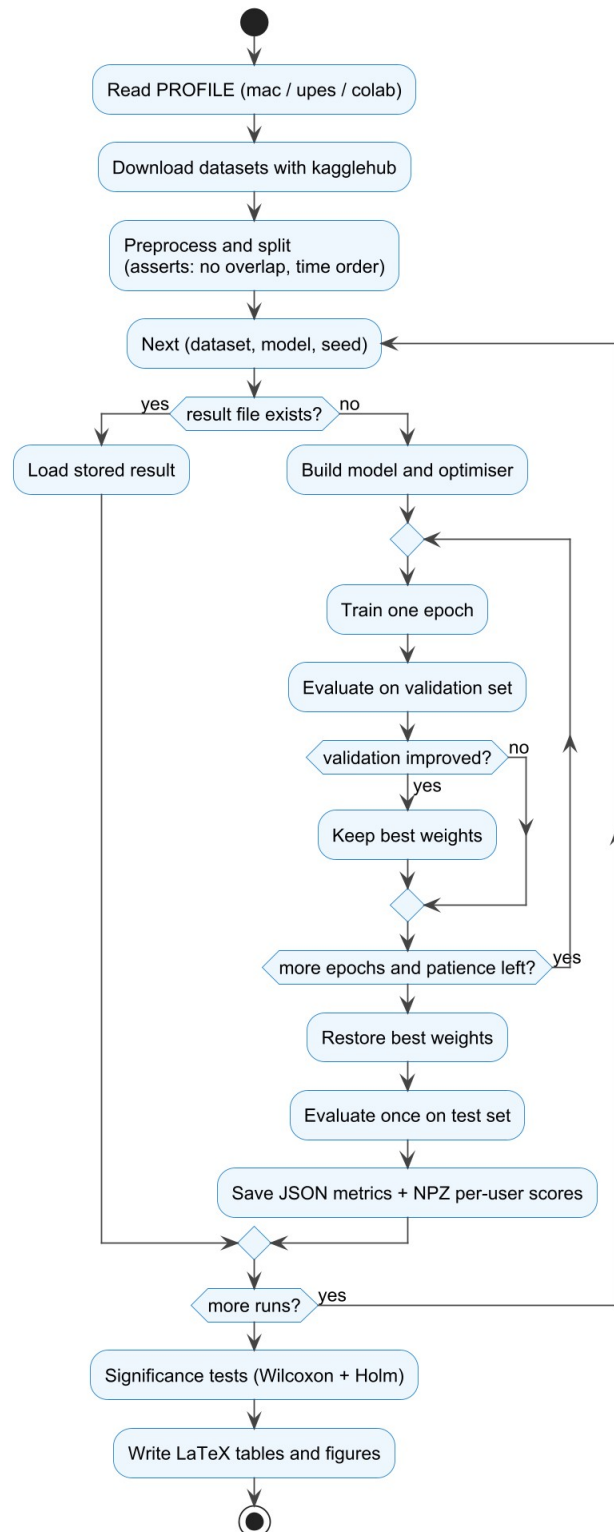


Figure 3: Activity diagram of one notebook run (Source: Authors' own work)

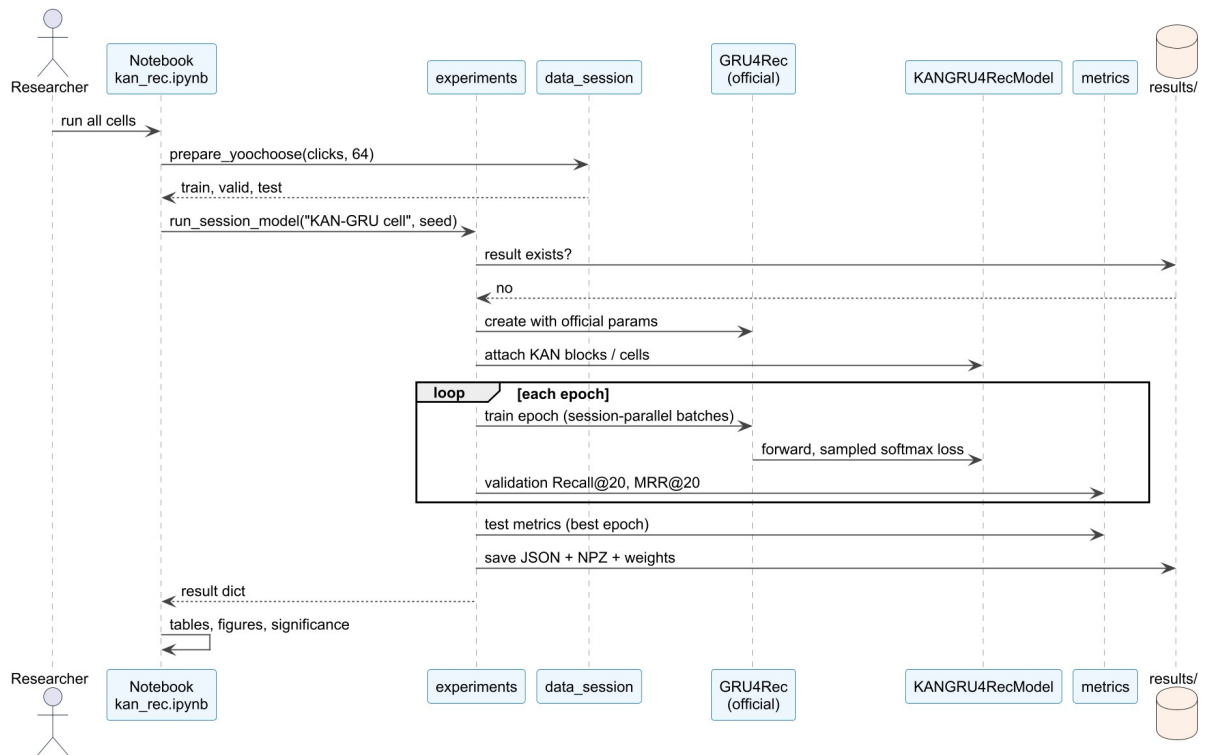


Figure 4: Sequence diagram for training one session model (Source: Authors' own work)

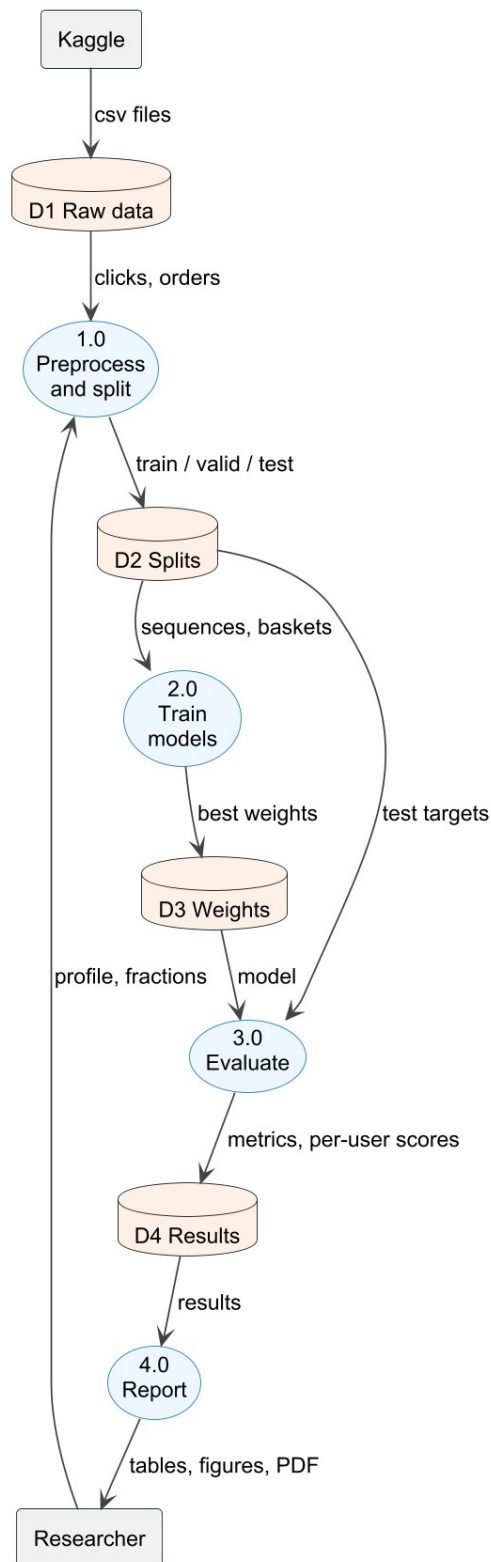


Figure 5: Level-1 data flow diagram (Source: Authors' own work)

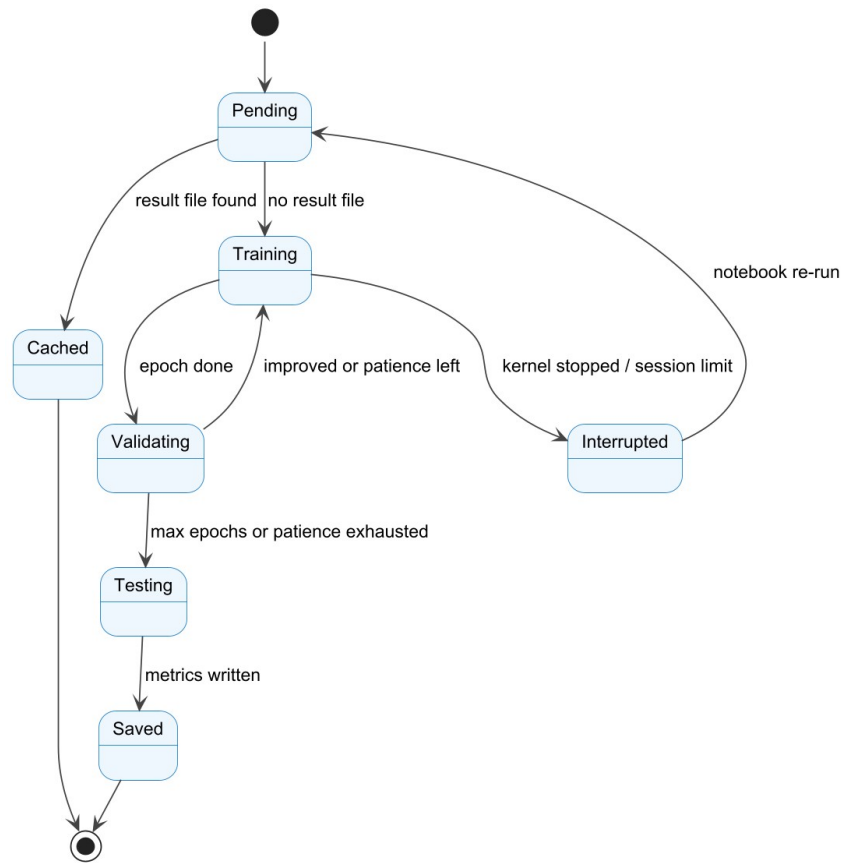


Figure 6: State diagram of one experiment run (Source: Authors' own work)

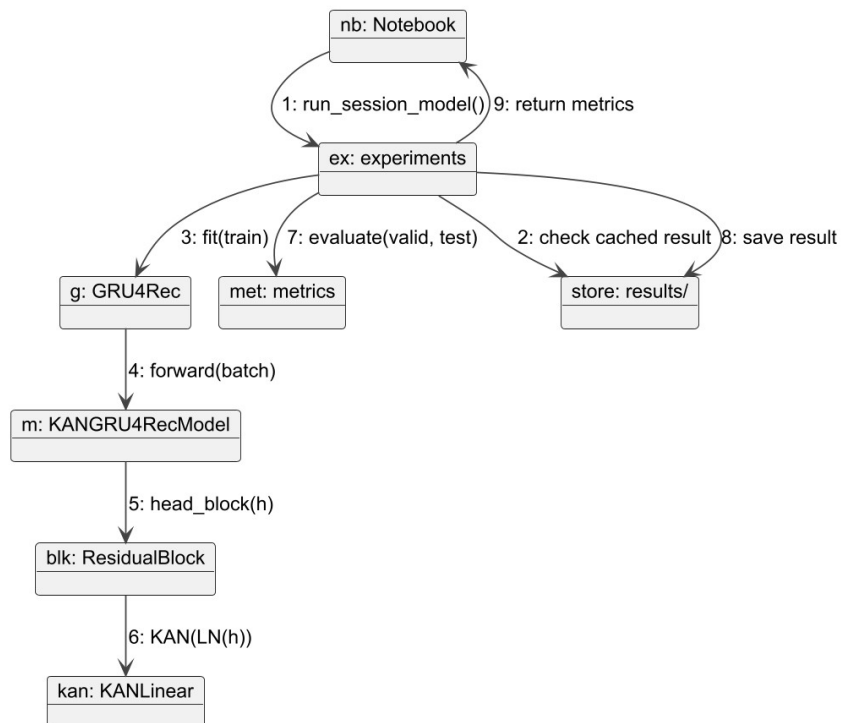


Figure 7: Collaboration (communication) diagram for a KAN-head run (Source: Authors' own work)

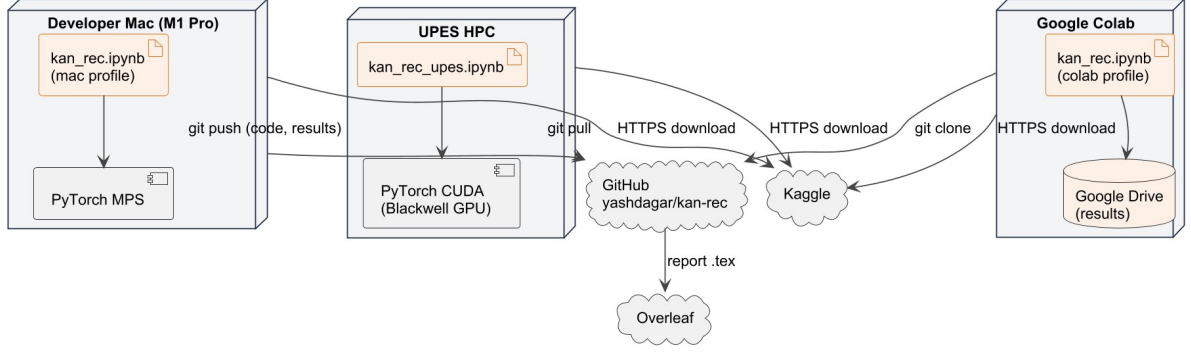


Figure 8: Deployment diagram (Source: Authors' own work)

2.8 Assumption and Dependencies

- The Kaggle datasets remain available with the same content; Instacart requires accepting the competition rules once.
- The official GRU4Rec repository remains available at the pinned commit.
- Published numbers (SR-GNN table, van Maasakkers et al.) are correct and use the protocols described.
- Test sets are large enough that single-seed differences above about 0.002 are meaningful; this is checked with significance tests and extra seeds.
- UPES HPC access and GPU time are available for the multi-seed runs.

3 SYSTEM REQUIREMENTS

3.1 User Interface

The system has no graphical application. Its user interface is the Jupyter notebook:

- a configuration cell with `PROFILE = "mac" | "upes" | "colab"`;
- progress logs per epoch (loss, validation metrics, seconds per epoch), also written to `results/<profile>/run.log`;
- inline tables and figures (results, significance, curves, learned KAN functions);
- command-line entry points: `uv run pytest, tools/build_notebook.py, tools/flatten_report.py`;
- the compiled PDF report and the presentations for evaluators.

3.2 Software Interface

Table 5: Module interfaces.

Module	Provides	Used by
kan.py	KANLinear, ResidualBlock, matched_mlp	session.py, basket.py, kannbr.py
data_session.py	load_yoochoose_clicks, prepare_yoochoose	notebook
data_basket.py	load_instacart, load_dunnhumby, BasketData	notebook, basket.py, kannbr.py
session.py	KANGRU4RecModel, KAN-GRU cell, train_session_model, evaluate	experiments.py
basket.py	BasketGRU, train_basket_gru, TopFreq, TIFU-KNN	experiments.py
kannbr.py	NBRFeaturizer, KANNBR, train_kannbr, shape_functions	experiments.py, notebook

metrics.py	session and basket metrics	session.py, basket.py, kannbr.py
experiments.py	run_* functions, load_results	notebook
report.py	LaTeX tables, Wilcoxon + Holm	notebook

Modules communicate through in-process Python calls; data are passed as NumPy arrays, pandas frames and PyTorch tensors. Each run_* function takes the dataset, a model name and a seed, returns a result dictionary, and writes it to results/<profile>/<dataset>/<model>__s<seed>.json. External services are reached only through kagglehub (downloads) and git (code and results).

3.3 Database Interface

No database management system is used. Persistent data are files: raw CSV files in data/ (git-ignored), cached preprocessed splits, model weights (.pt, git-ignored), and one JSON + NPZ pair per run. The JSON record holds the dataset, model, seed, configuration, per-epoch history, best epoch, validation and test metrics, parameter count and timing; the NPZ holds per-prediction or per-user metric values for significance tests. File existence is the resume key.

3.4 Protocols

- HTTPS with an API token for Kaggle downloads (kagglehub); about 2 GB in total.
- Git over HTTPS for GitHub; results are pushed after each batch of runs.
- No network access during training or evaluation; runs are synchronised only through result files.
- On Colab, results are written to Google Drive so an interrupted session resumes.

4 NON-FUNCTIONAL REQUIREMENTS

4.1 Performance Requirements

Table 6: Measured cost and accuracy of key models.

Model (Mac, seed 42)	Parameters	s / epoch	Test metric
GRU4Rec	9,689,264	78.5	Recall@20 0.6901
KAN-GRU cell (rational)	10,148,704	73.3	Recall@20 0.7095
MLP-GRU cell (matched)	10,149,264	63.0	Recall@20 0.7035
KAN-GRU cell (B-spline)	14,756,624	110.3	Recall@20 0.7007
Basket-GRU (Dunnhumby)	12,205,186	8.7	Recall@10 0.1663
KAN-NBR (Dunnhumby)	64	0.9	Recall@10 0.2101
Basket-GRU (Instacart 10%)	13,935,386	75.5	Recall@10 0.2453

- A YooChoose 1/64 epoch must finish within 2 minutes on the Mac and evaluation of all 55,357 test predictions within 5 seconds.
- Peak memory must stay below 12 GB on the 16 GB Mac (chunked evaluation, memory freed after runs).
- Every KAN model must report parameters, seconds per epoch and inference time next to accuracy.

- The UPES profile must finish three seeds of the key models within one day of GPU time.

4.2 Security Requirements

- The datasets are anonymised by their publishers; no personal data are collected or linked.
- Raw data are not redistributed: data/ is git-ignored, in line with the Kaggle terms.
- Credentials (Kaggle token, GitHub token) are never stored in the repository or the notebooks.
- The official GRU4Rec code is cloned, not committed, respecting its licence.
- Validation: unit tests, leakage assertions in every pipeline, and a reproduction check against published results.

4.3 Software Quality Attributes

Table 7: Software quality attributes.

Attribute	How it is met
Correctness	41 unit tests; wrapper output identical to official GRU4Rec; split assertions
Reliability / robustness	resumable runs; zero-initialised residual blocks; stable learning rates
Availability	public GitHub repository; Kaggle data; Colab profile needs no local hardware
Adaptability / flexibility	new bases, placements or datasets added as one function or dictionary entry
Interoperability	standard file formats (JSON, NPZ, CSV, LaTeX, PDF)
Maintainability	small modules with one role each; notebooks generated from one builder script
Portability	runs on macOS (MPS), Linux (CUDA) and Colab; pure Python
Reusability	KAN layer and metrics usable independently of the notebook
Testability	pure functions for metrics and features; pytest suite
Usability	one profile variable; one command to run; tables written directly into the report

5 OTHER REQUIREMENTS

- **Reproducibility:** fixed seeds, pinned third-party commit, recorded configuration in every result.
- **Fairness of comparison:** every KAN variant has a parameter-matched MLP control; the decision rule is applied on validation data only.
- **Documentation:** README with instructions per profile; report with errata appendix.
- **Ethics:** results are reported whether or not they favour KANs.

APPENDIX A: GLOSSARY

Term	Meaning
KAN	Kolmogorov-Arnold Network: learnable univariate function on every edge
MLP	multi-layer perceptron with fixed activations
GRU / GRU4Rec	gated recurrent unit / GRU-based session recommender
Basket-GRU	GRU over multi-hot baskets with a sigmoid output per product
KAN-NBR	additive KAN over repeat-purchase features for next-basket recommendation
RBF, B-spline, Chebyshev, GR-KAN	basis families used inside KAN layers
Recall@K / HR@K	share of targets ranked in the top K
MRR@K	mean reciprocal rank, zero when the target is outside the top K
NDCG@K	normalised discounted cumulative gain
P@ B	precision at the size of the true basket
TIFU-KNN	time-decayed item frequency with nearest-neighbour users
MPS	Apple Metal Performance Shaders backend of PyTorch
Wilcoxon, Holm	paired signed-rank test; multiple-comparison correction
SRS	Software Requirements Specification

APPENDIX B: ANALYSIS MODEL

The analysis model is a controlled comparison. For each dataset, every variant shares the baseline's hyperparameters; a KAN placement is called helpful only if, on validation data, it beats both the baseline and its parameter-matched MLP. Table 8 lists the current test results for these pairs.

Table 8: KAN placements against parameter-matched MLPs (test, seed 42).

Dataset	Placement	Metric	KAN	Matched MLP
YooChoose	Head	Recall@20	0.6832	0.6878
YooChoose	Input	Recall@20	0.6669	0.6854
YooChoose	Cell (RBF)	Recall@20	0.6926	0.7016
YooChoose	Cell (rational)	Recall@20	0.7095	0.7035
Instacart	Head	Recall@10	0.1420	0.2375
Instacart	Input	Recall@10	0.1067	0.2293
Instacart	Cell	Recall@10	0.2517	0.2639
Dunnhumby	Head	Recall@10	0.1535	0.1722
Dunnhumby	Input	Recall@10	0.1634	0.1787
Dunnhumby	Cell	Recall@10	0.1709	0.1726

APPENDIX C: ISSUES LIST

#	Issue	Status
1	Third seed (44) for the four key YooChoose models; seed 43 for the matched MLP-GRU cell	Open
2	Full Instacart and yoochoose1/4 runs on the UPES GPU	Open
3	Instacart clustering gives 12,058 groups vs 9,407 in the original paper	Documented
4	Benchmark of torch.compile and float32 optimisations on the GPU	Open
5	Results, conclusion and abstract chapters of the report	In progress
6	Publication of the KAN-GRU cell and KAN-NBR results	Planned